

Module 3: AI Tools

Follow-up

Filip Kronström

filipkro@chalmers.se

February 13, 2025

A look at your **assignments**

Assignment 3.

- Discussion:
- In most cases, the accuracy of a system when deployed is **lower** than the accuracy measured during development (training).
- What are **possible reasons** for this? Discuss which **assumptions** are made during development that **may not hold**, and what could be done to improve the system.
- Was this the case for this assignment?

Notes from your hand-ins

Potential issues

- Preprocessing may change
- Overfitting
- Distributional shift between cities, label shift
- Non-stationarity, changes with time
- Lack of monitoring
- Choice of metric (e.g., accuracy vs F1)
- Imbalanced data

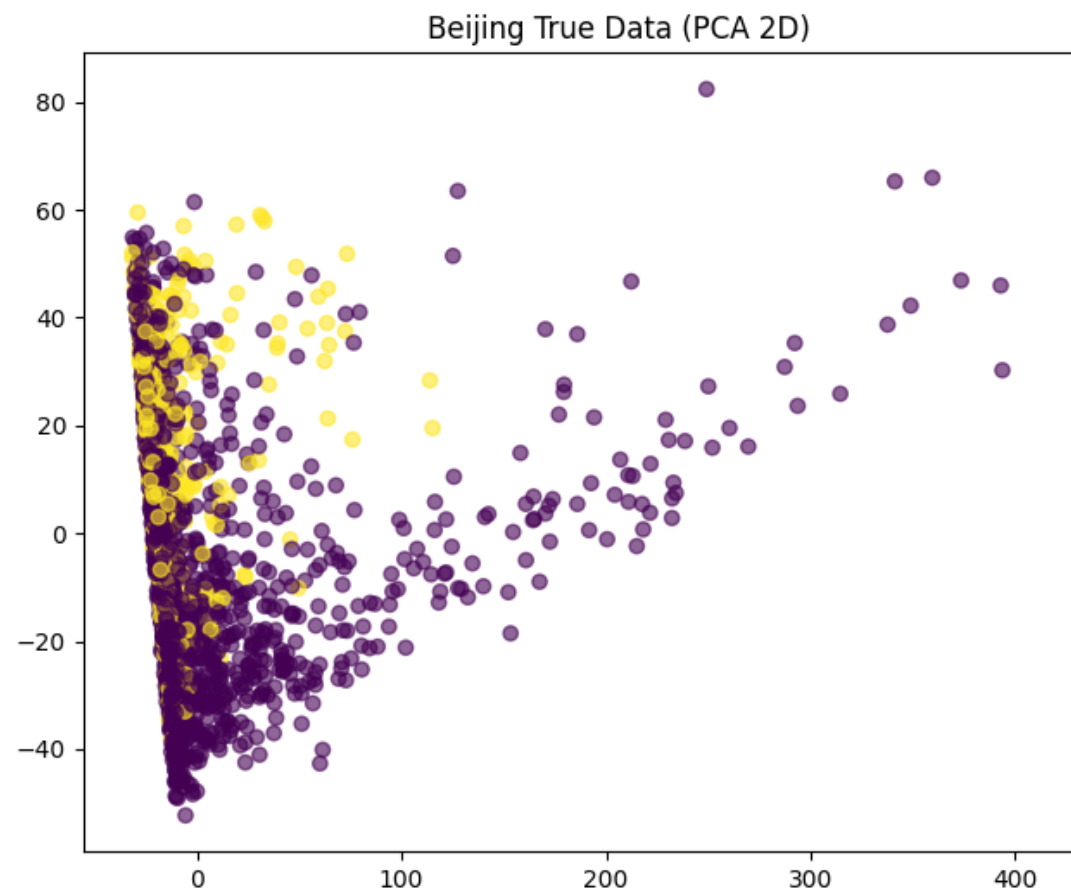
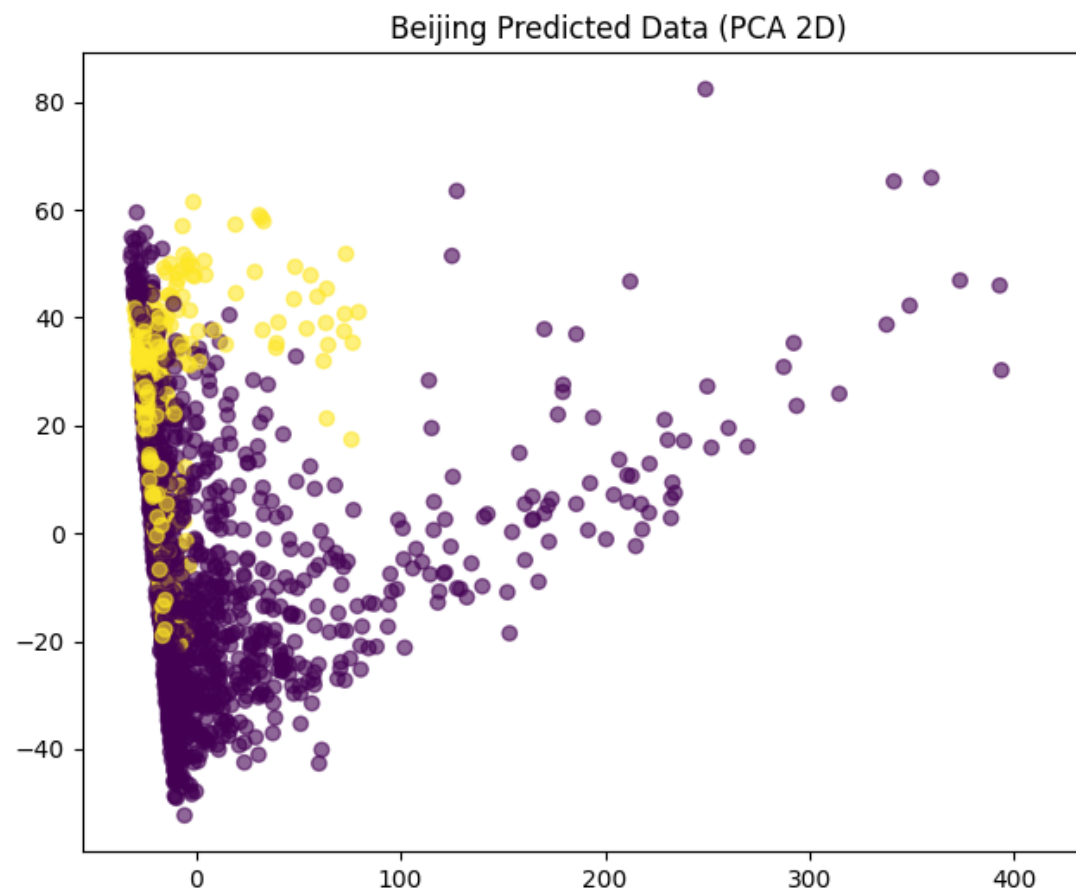
Suggested solutions

- Additional checks, bookkeeping
- Gathering more diverse data (e.g., cities)
- Domain-specific (city-specific) models
- Updating model over time
- Improved model
- Over/under-sampling, weighting
- Remove “shifting” features
- Detect change, adapt model
- Adapt hyperparameters

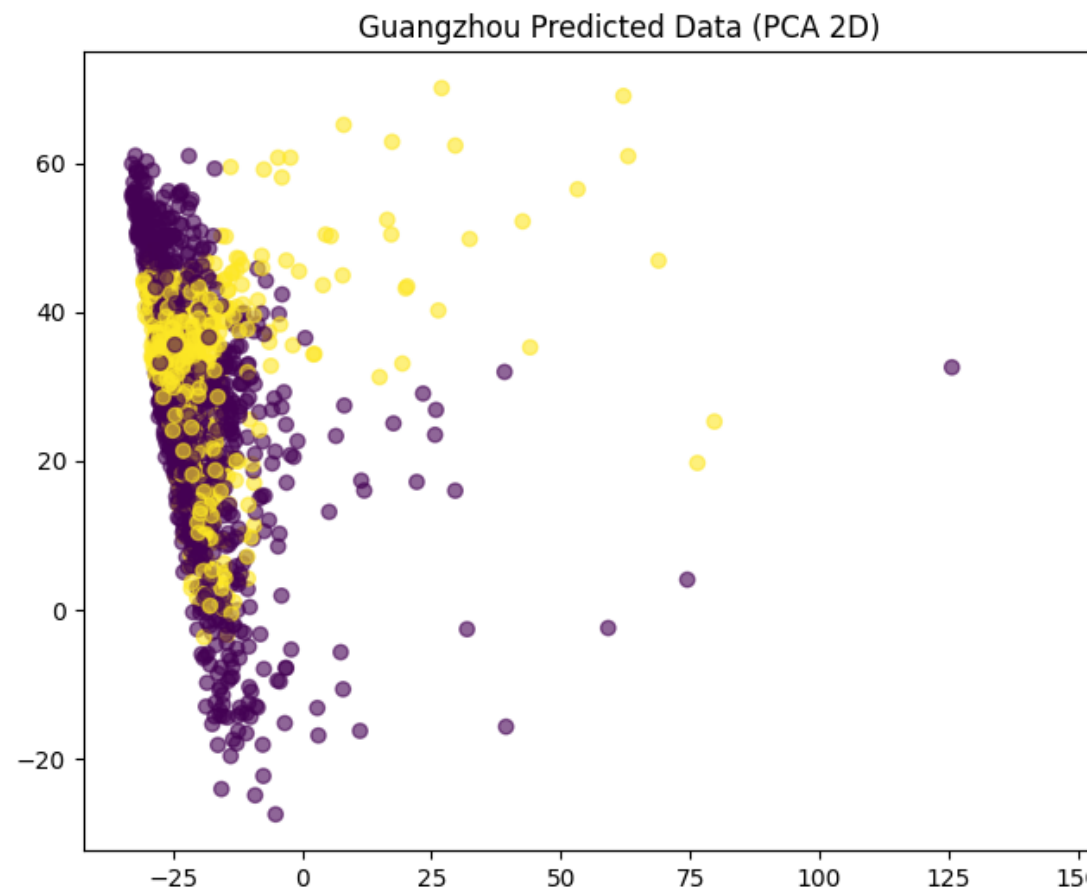
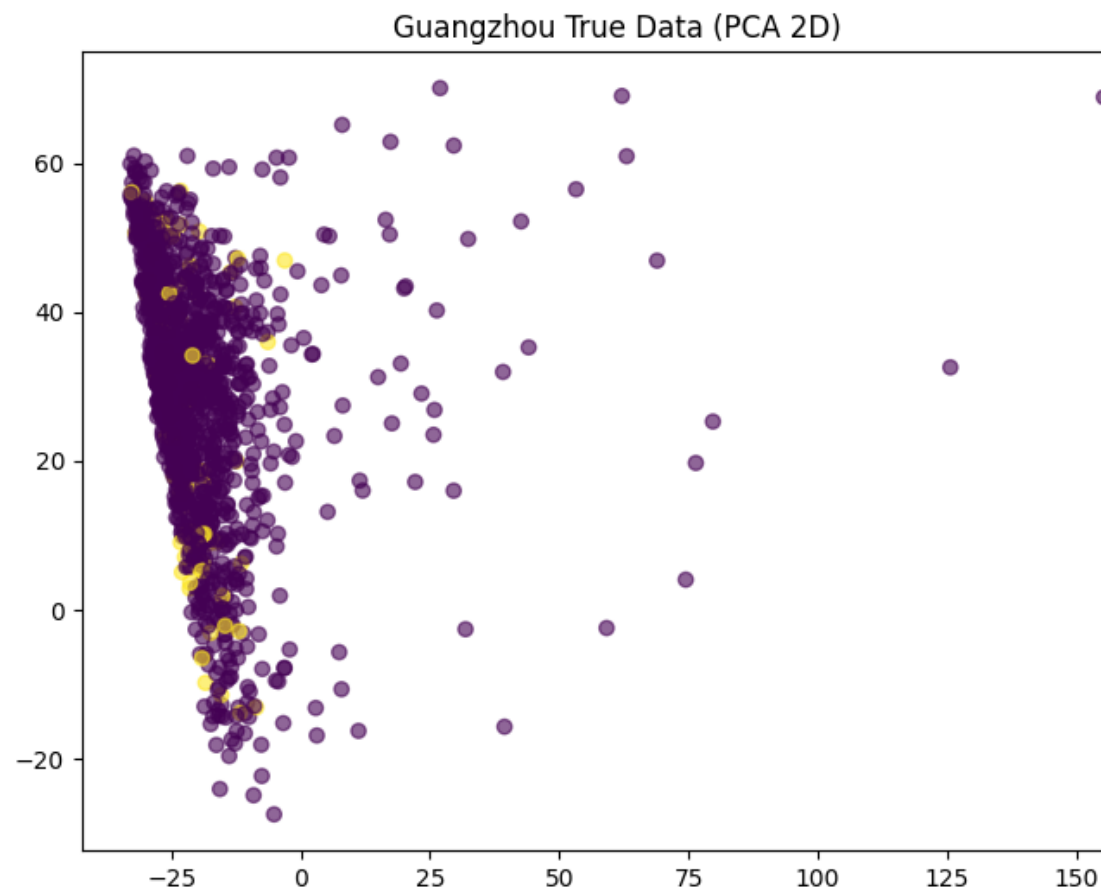
Notes from your hand-ins

- Common patterns
 - No **mathematical notation**. Try to make your ideas precise
 - No **examples**. When specifically does something work/not?
 - No **specifics** regarding distributional shift, data drift, etc.
 - No **specifics** regarding assumptions.
- Let's try to improve this!

Issues with the model



Issues with the model

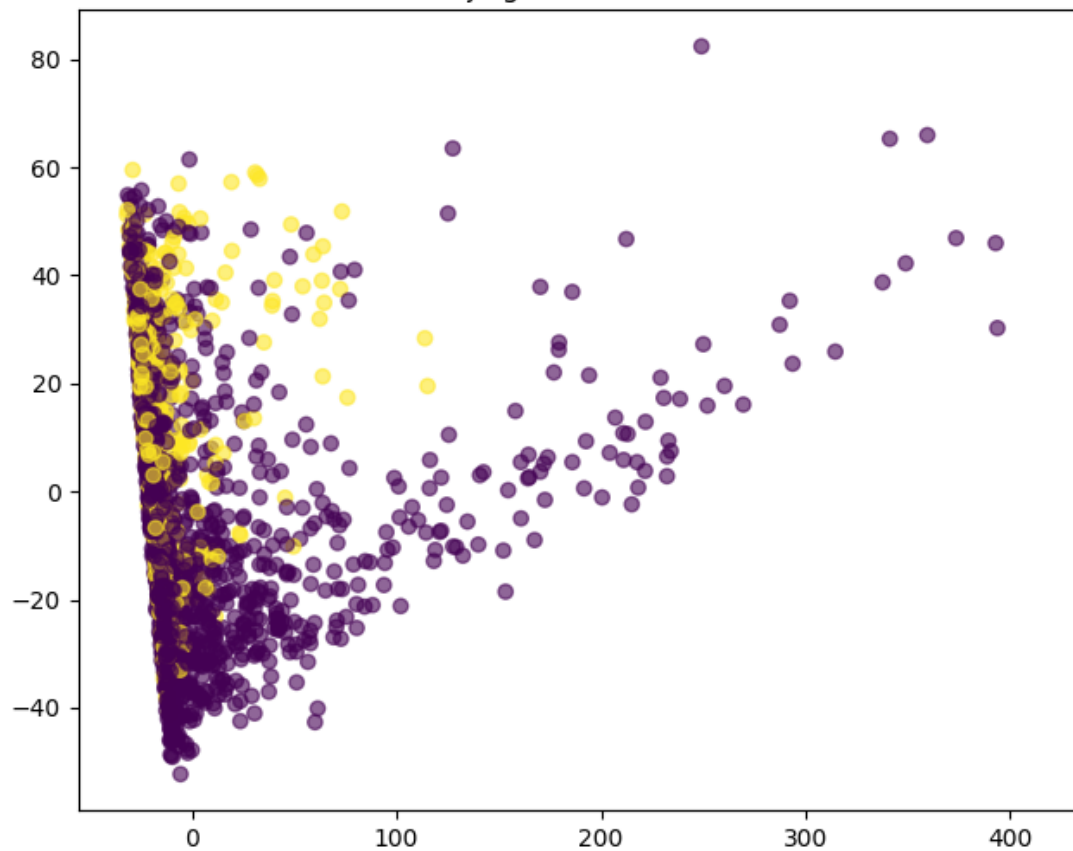


Issues with the model

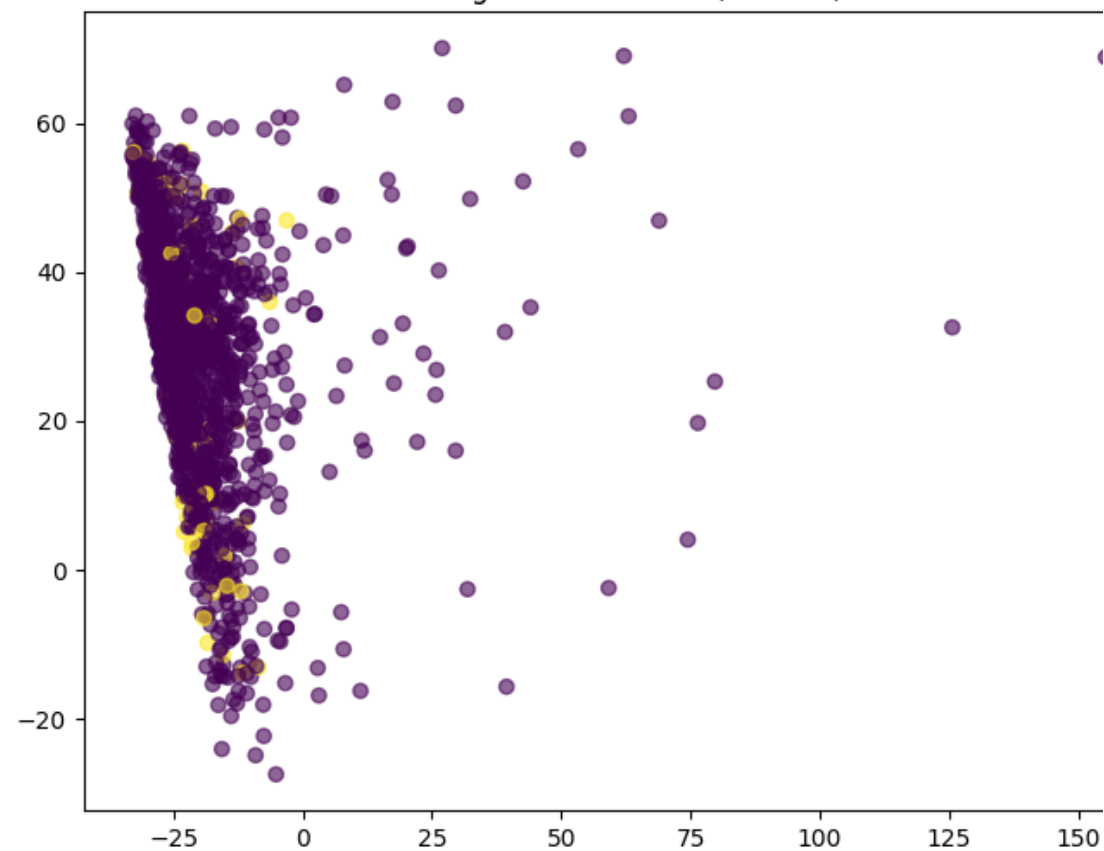


Issues with the data

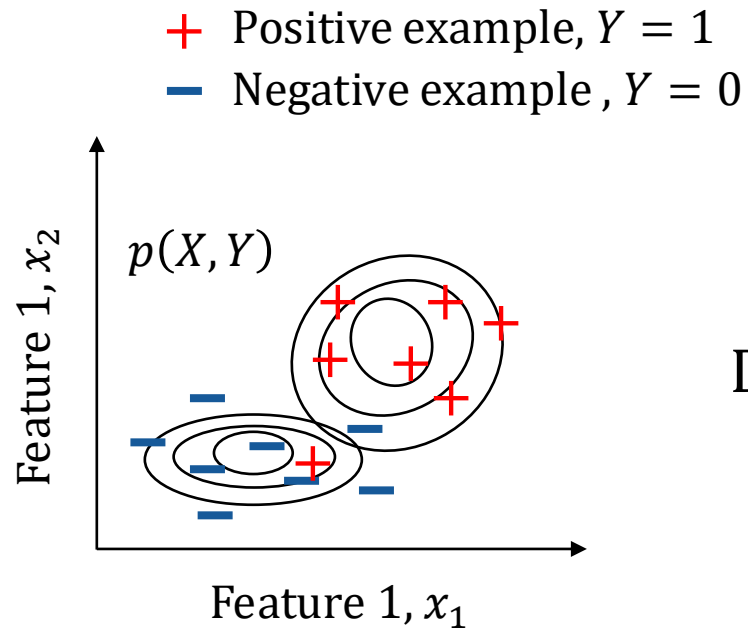
Beijing True Data (PCA 2D)



Guangzhou True Data (PCA 2D)

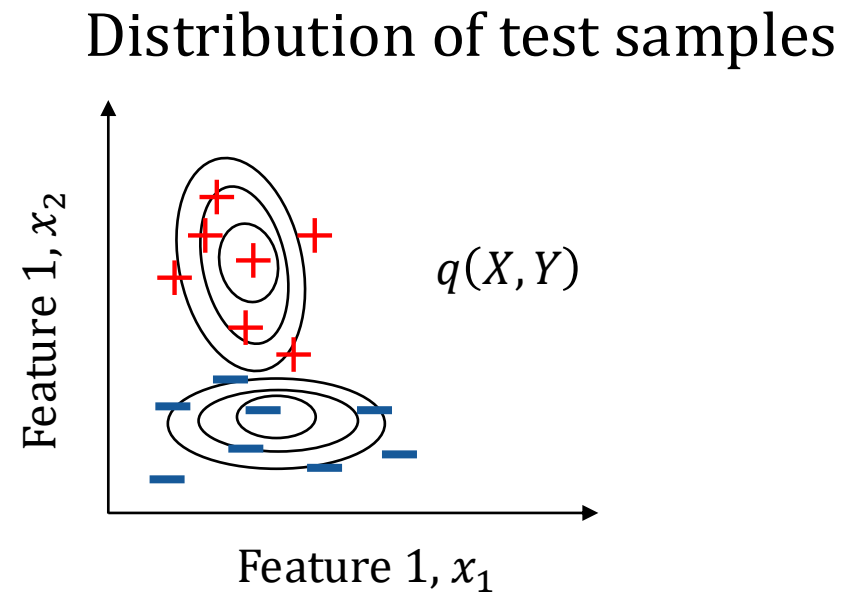
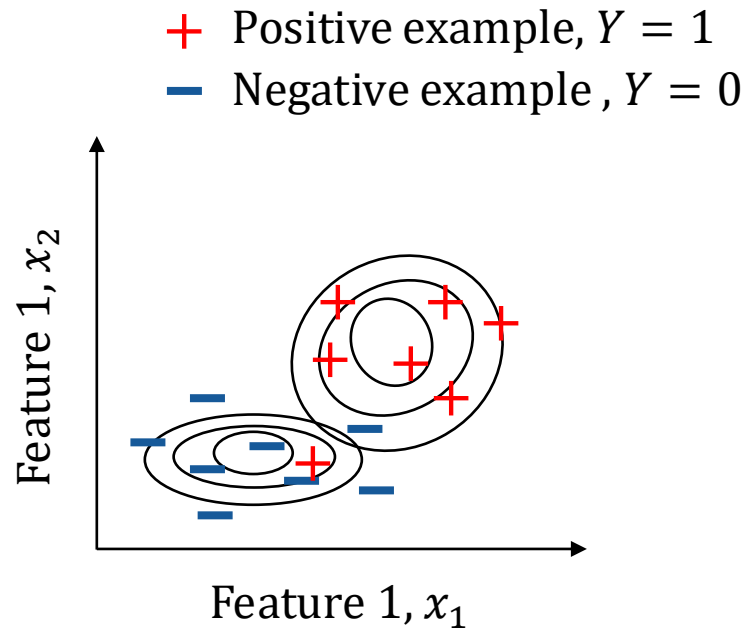


Example: Distributional shift



Distribution of training samples

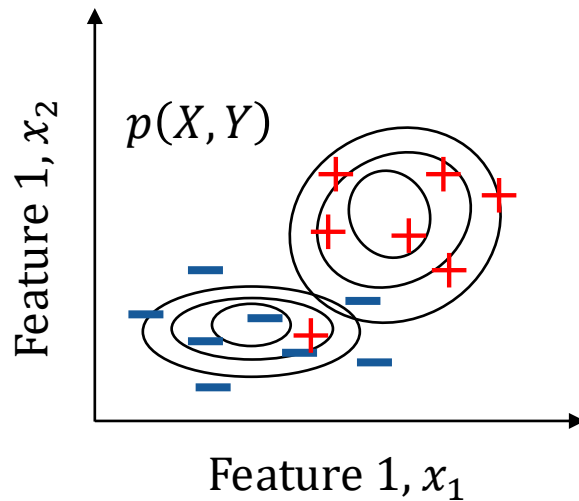
Example: Distributional shift



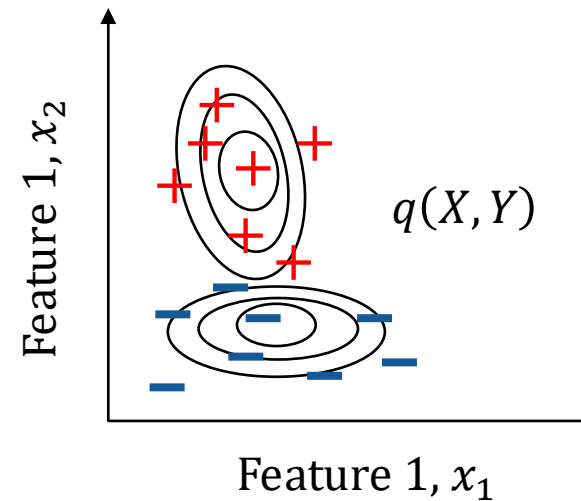
Example: Distributional shift

- Training samples $(X_i, Y_i) \sim p(X, Y)$, test $(X_i', Y_i') \sim q(X, Y)$

$$p(X, Y) \neq q(X, Y)!$$



Good classifier

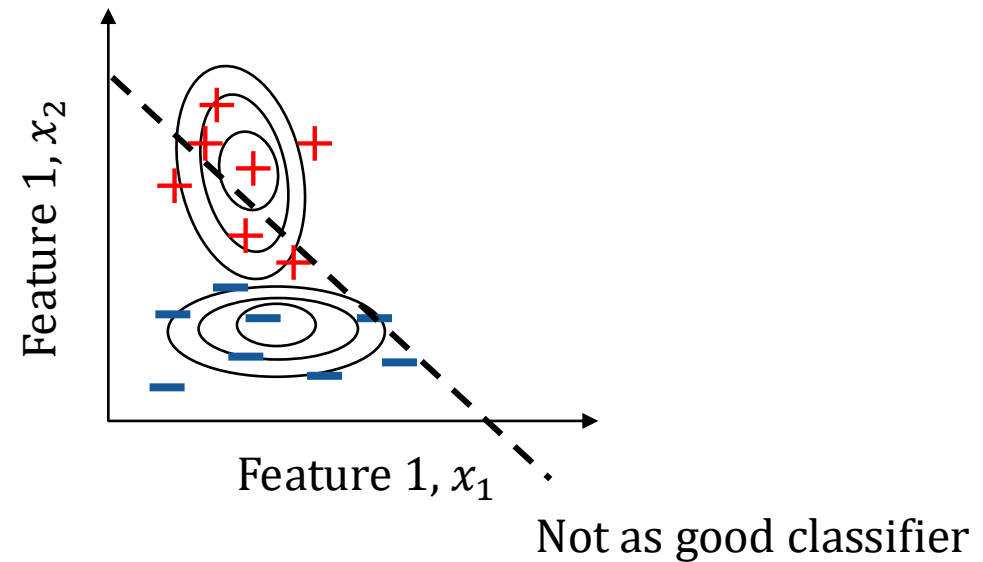
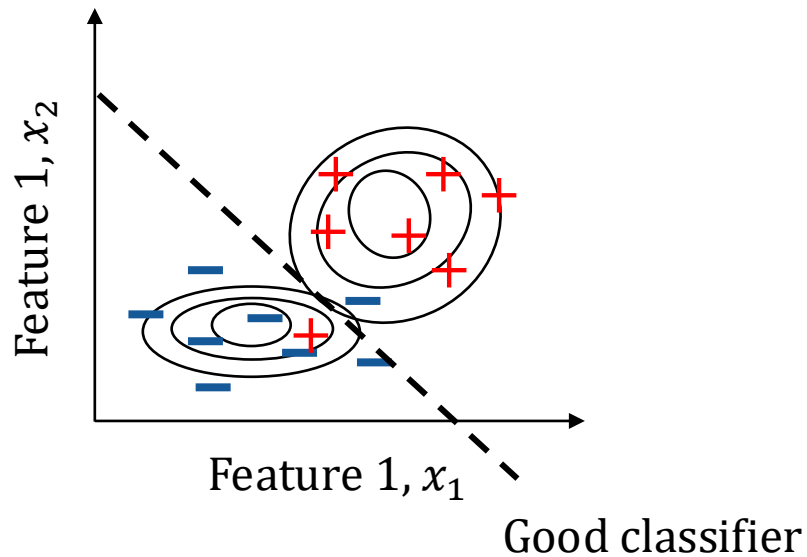


Not as good classifier

Example: Distributional shift

- Training samples $(X_i, Y_i) \sim p(X, Y)$, test $(X_i', Y_i') \sim q(X, Y)$

$$p(X, Y) \neq q(X, Y)!$$

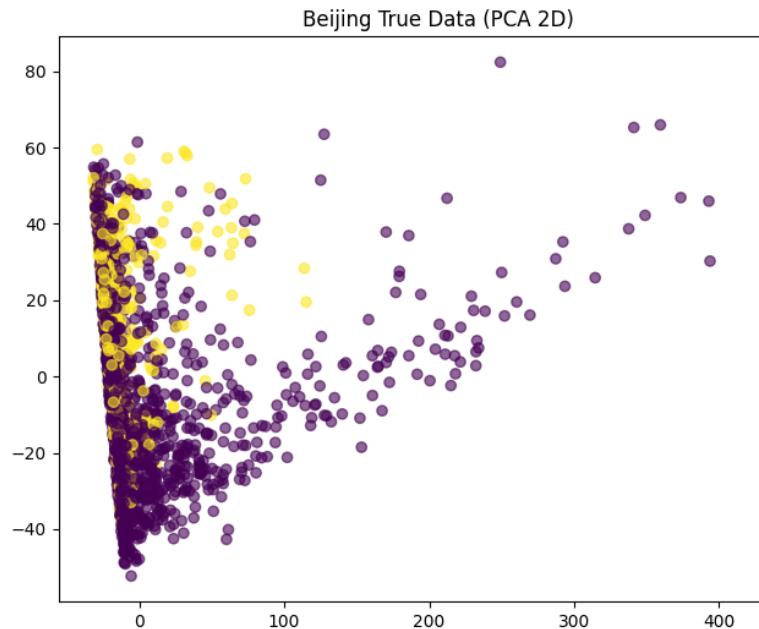


Example: Assumptions we often make

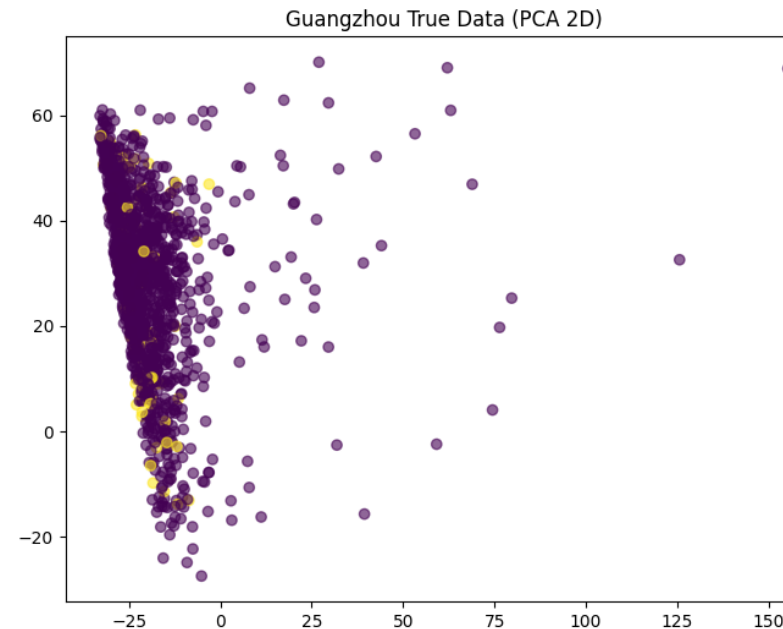
- Training samples $(X_i, Y_i) \sim p(X, Y)$, test $(X_i', Y_i') \sim q(X, Y)$
- **Assumption: (Identical distributions)**

$$p(X, Y) = q(X, Y) \quad \text{or} \quad p(Y | X) = q(Y | X)$$

Same joint distribution



Same labeling function



Solve in groups

- Pick one issue and one solution (not necessarily matching)
 - Describe both (or at least the left one) **mathematically**
 - Describe **tools** that can help

Potential issues

- Preprocessing may change
- Overfitting
- ~~• — Distributional shift between cities, label shift~~
- Non-stationarity, changes with time
- Lack of monitoring
- Choice of metric (e.g., accuracy vs F1)
- Causality/correlation

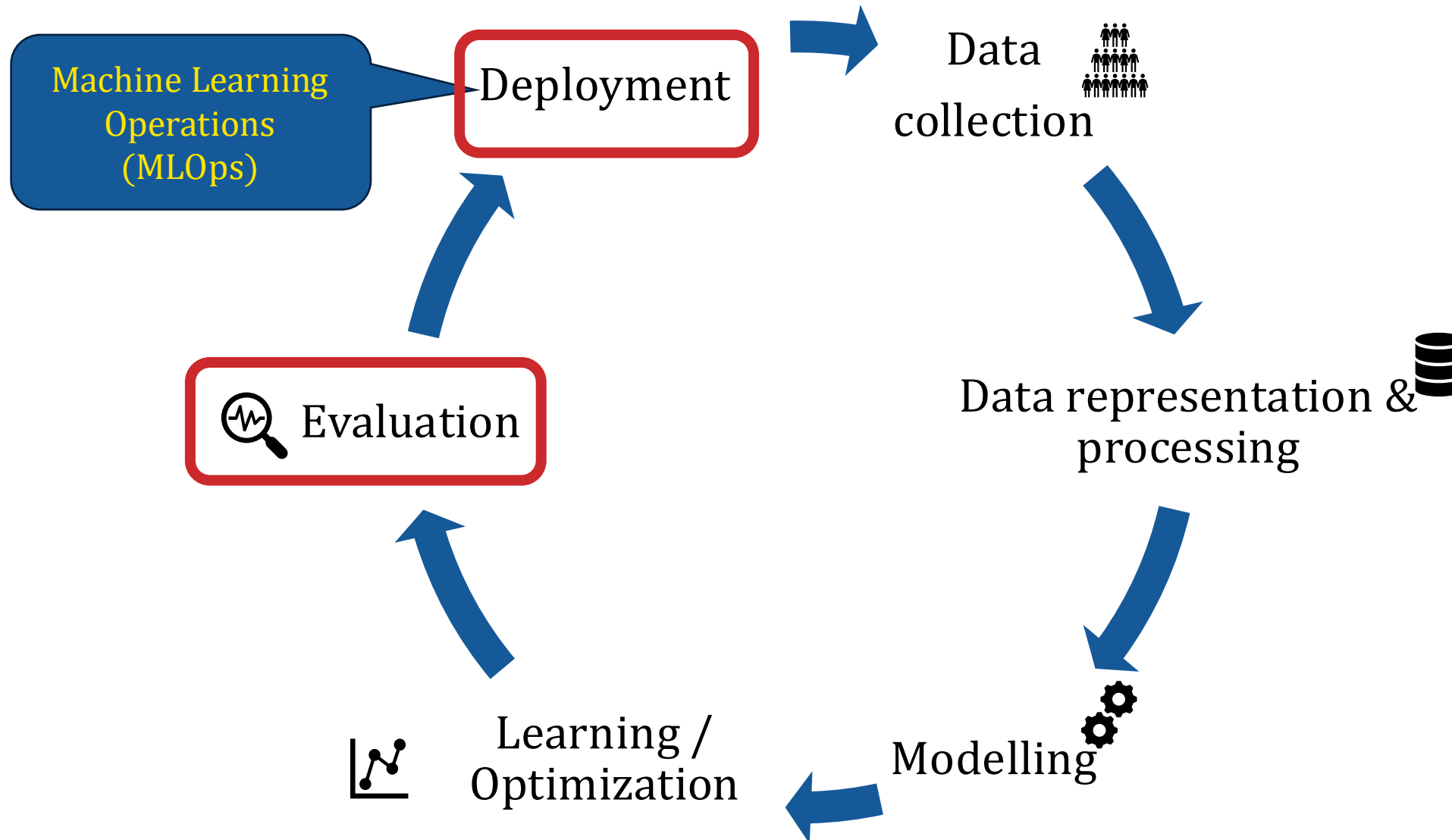
Suggested solutions

- Additional checks, bookkeeping
- Gathering more diverse data (e.g., cities)
- Updating model over time
- Domain-specific (city-specific) models
- Improved model
- Over/under-sampling, weighting
- Remove “shifting” features

Assignment 3.

- **Implementation:** Most of you solved the problem well! Good job!
- What can be improved:
 - **Description** of your solution:
 - How did you tune hyperparameters?
 - How did you evaluate your problem?
 - What choices did you make re: distance?
 - **Autonomy** in problem solving—relying less on “checkbox” questions

AI tools in ML life cycle



3. **Model** evaluation

Recap: Mean Squared Error (MSE)

We can measure its error using the **mean squared error**

	Mean	Error	Square
--	------	-------	--------

$$\hat{R}(\theta) = \frac{1}{n} \sum_{i=1}^n (\theta^\top x_i - y_i)^2$$

n = number of data points

Recap: Classification metrics

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

$$F1 = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

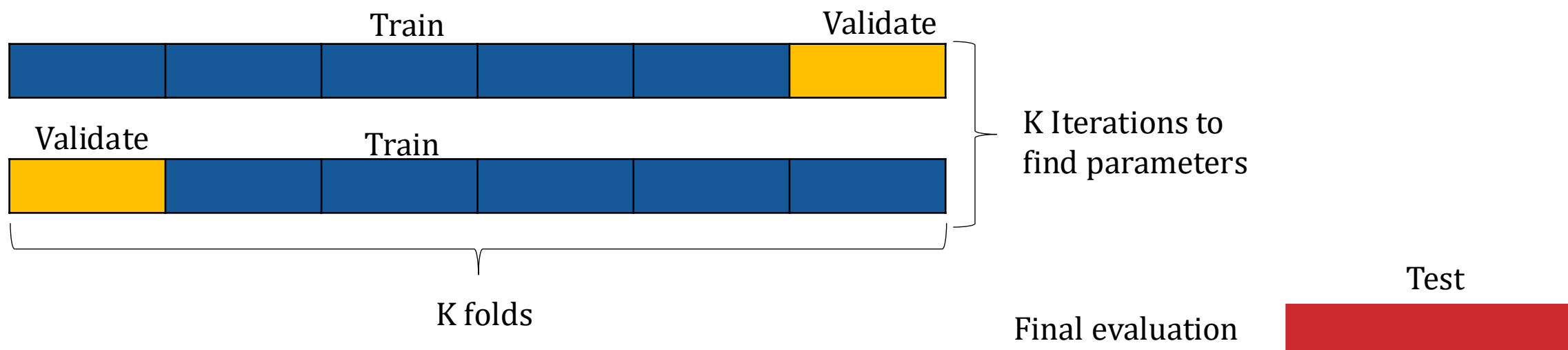
$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

		Actual	
		Positives (1)	Negatives (-1)
Predicted	Positives (1)	True Positive	False Positive
	Negatives (-1)	False Negative	True Negative

Recap: (K-folds) Cross Validation (CV)

1. Divide the samples into k parts
2. Use $k-1$ of the parts for training, and 1 for validation
3. Repeat the procedure k times, rotating the validation set
4. Determine an expected performance metric (e.g. MSE) based on the results across the iteration



Reporting performances for subpopulations

How do we use **group attributes** (e.g., sex, age, ethnicity) in predictions?

Using group attributes can result in worse subgroup performance while improving overall model performance.

Example: logistic regression trained to predict sleep apnea with sex and age is **worse** for younger male and older female patients.

Reporting of **performances for subpopulations** can lead to **better informed user.**

GROUP g	SIZE n_g	ERROR RATE		GAIN
		$R(h_0)$	$R_g(h_g)$	$\Delta_g(h_g, h_0)$
female, <30	48	38.1%	26.8%	11.3%
male, <30	49	23.9%	26.7%	-2.8%
female, 30 to 60	307	30.3%	29.1%	1.2%
male, 30 to 60	307	15.4%	15.2%	0.2%
female, 60+	123	19.3%	21.9%	-2.6%
male, 60+	181	11.0%	8.2%	2.8%
Total	1152	20.4%	19.4%	1.0%

What is Interpretability in ML models?

Most agree it is something different from performance.

This course: Ability to explain or to present in understandable terms to a human (Doshi-Velez, 2017)

- Context-depend

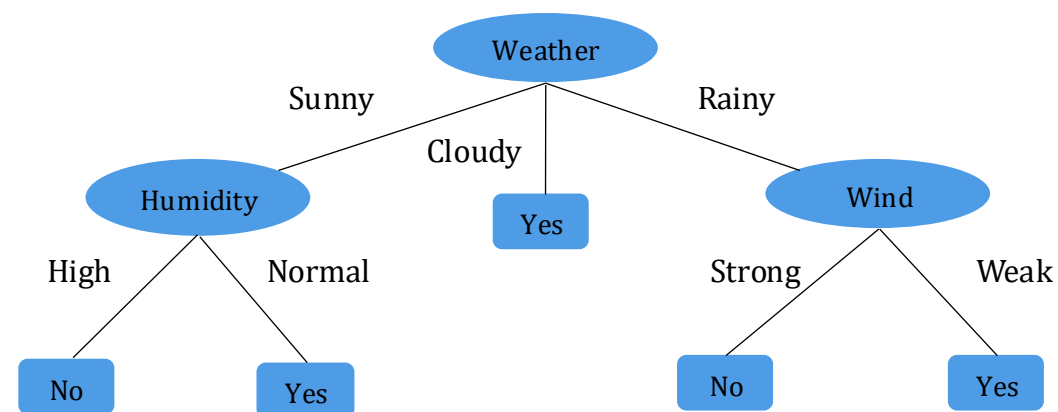
What is interpretable in ML models?

- Interpretable models & explanations
 - rules, prototypes, linear models, saliency maps etc.
- Inherently interpretable models $\neq$ Explanations of black box models

Linear models

$$Y_i = \beta_0 + \beta_1 X_1 + \epsilon_i$$

Dependant Variable Linear Component Random Error



Decision trees

4. Machine Learning Operations (MLOps)

Discuss

Stop and think for a moment.

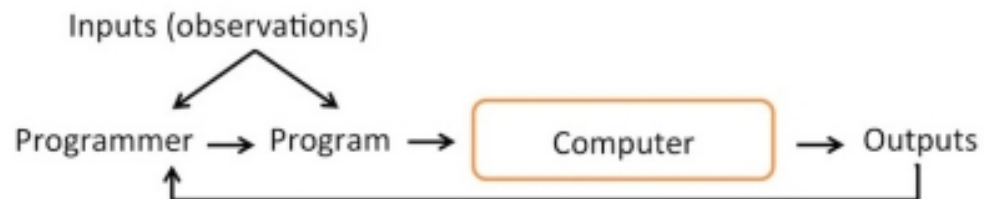
Which tools have you used to develop AI/ML systems already?

AI in deployment

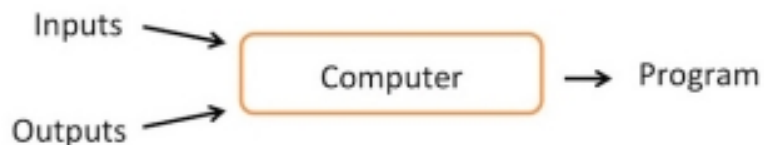
- Compared to research use, far fewer tools for production/deployment
- **Less established standards**
 - Also, huge diversity in needs!
 - E.g., teams, products, amount of data, data types
 - Compare, e.g., search ranking to medical diagnosis.

Machine learning as software

The Traditional Programming Paradigm



Machine Learning



Andrew Ng ✓
@AndrewYNg

1/The rise of Software Engineering required inventing processes like version control, code review, agile, to help teams work effectively. The rise of AI & Machine Learning Engineering is now requiring new processes, like how we split train/dev/test, model zoos, etc.

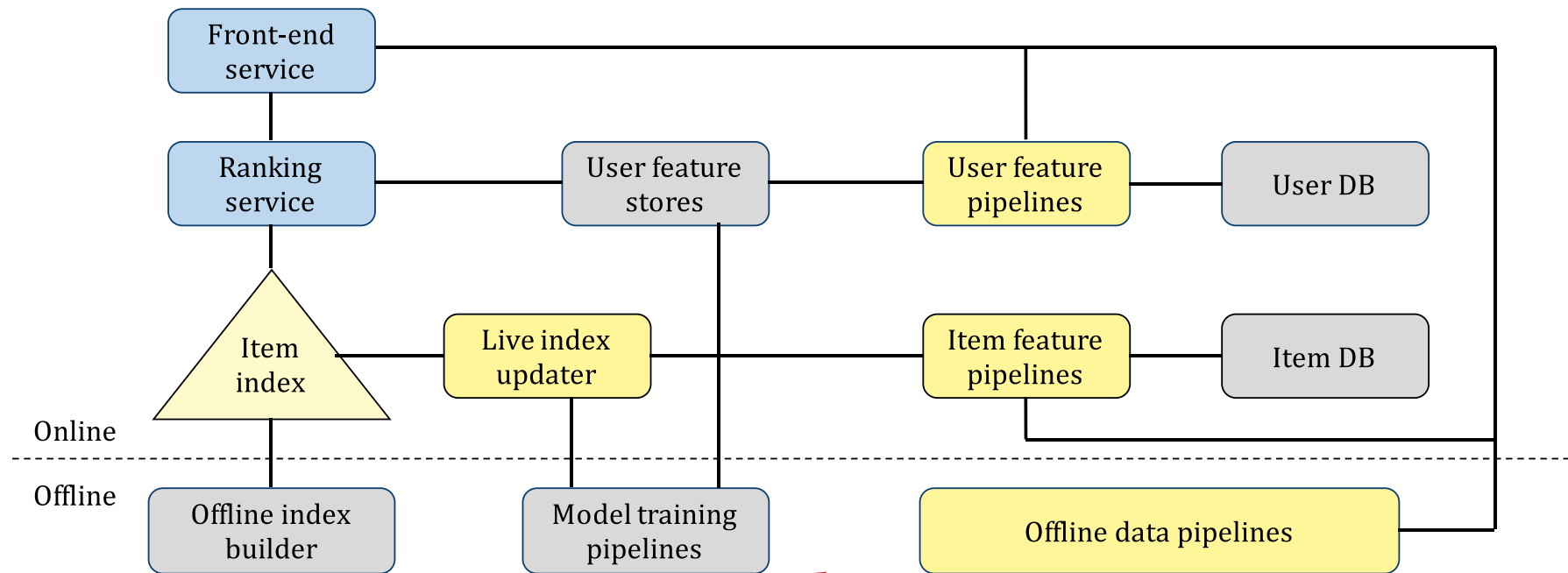
6:59 PM · Jan 3, 2019 · [Twitter Web Client](#)

Lessons from software engineering

- Much like traditional software, AI software need **maintenance**
- You read about this in the **assigned paper**
- When software is used **in production**, we expect
 - Version control
 - Modularity, backwards compatibility
 - Code review, bug-fixes
 - Robustness to external changes, ... and many more things

Machine learning in software

- Did you see this picture from LinkedIn in the additional material?



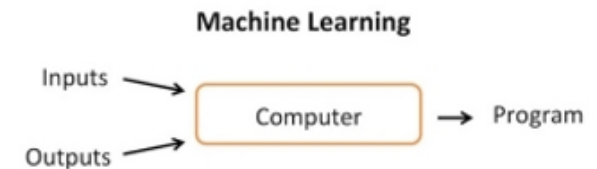
Machine learning algorithm

AI as software

- AI systems primarily change for two reasons
 1. Changes to **model pipeline** (e.g., objective/training algorithm/hyperparameters)
 2. Changes to **training data** (e.g., new customers/domains)
- These are **intimately connected!**
 - E.g., the best parameters for two datasets may not be the same
 - But they shouldn't be treated as one

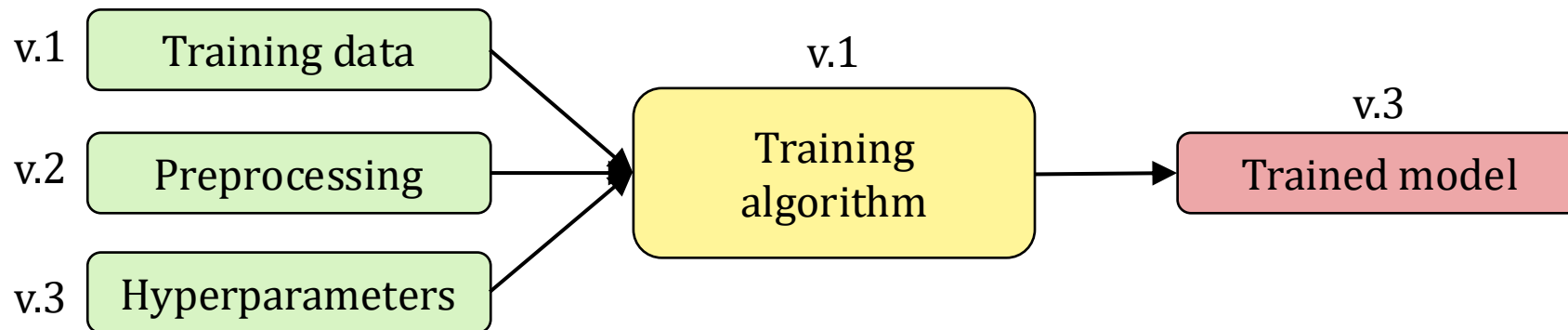
L1. Model versioning

- A machine learning model can be described in **several ways**
 1. The code of the program used to **train it**
 2. The learned **parameters**
- Both of the above are **insufficient on their own**
 - Code is “indifferent” to training data—not enough for application
 - Parameters are enough for application, but not for retraining



L1. Model versioning

- Use e.g., Git to version-control **code**—what about the model?
- Trained models have **dependencies**, like conventional software



- For models to be reproducible, **all of these** need versioning!

L2. Data versioning

- In development, it is common that data is changed or updated
- **Example:** A patient registry follows subjects with rheumatoid arthritis, gathering data through questionnaires. Every year,
 - New patients are added—New **observations**!
 - Questions are added and removed to the questionnaire—New **features**!
- How can we keep track of these?

Two approaches

Maintain releases

- Regularly **release** new “named” versions of data:
patient_data_1219.v2
- Only train models on these releases
- **Drawback:** Large overlap!

Maintain changes

- Log **queries** executed on a database, much like code versioning*
- Train models on specific version/commit

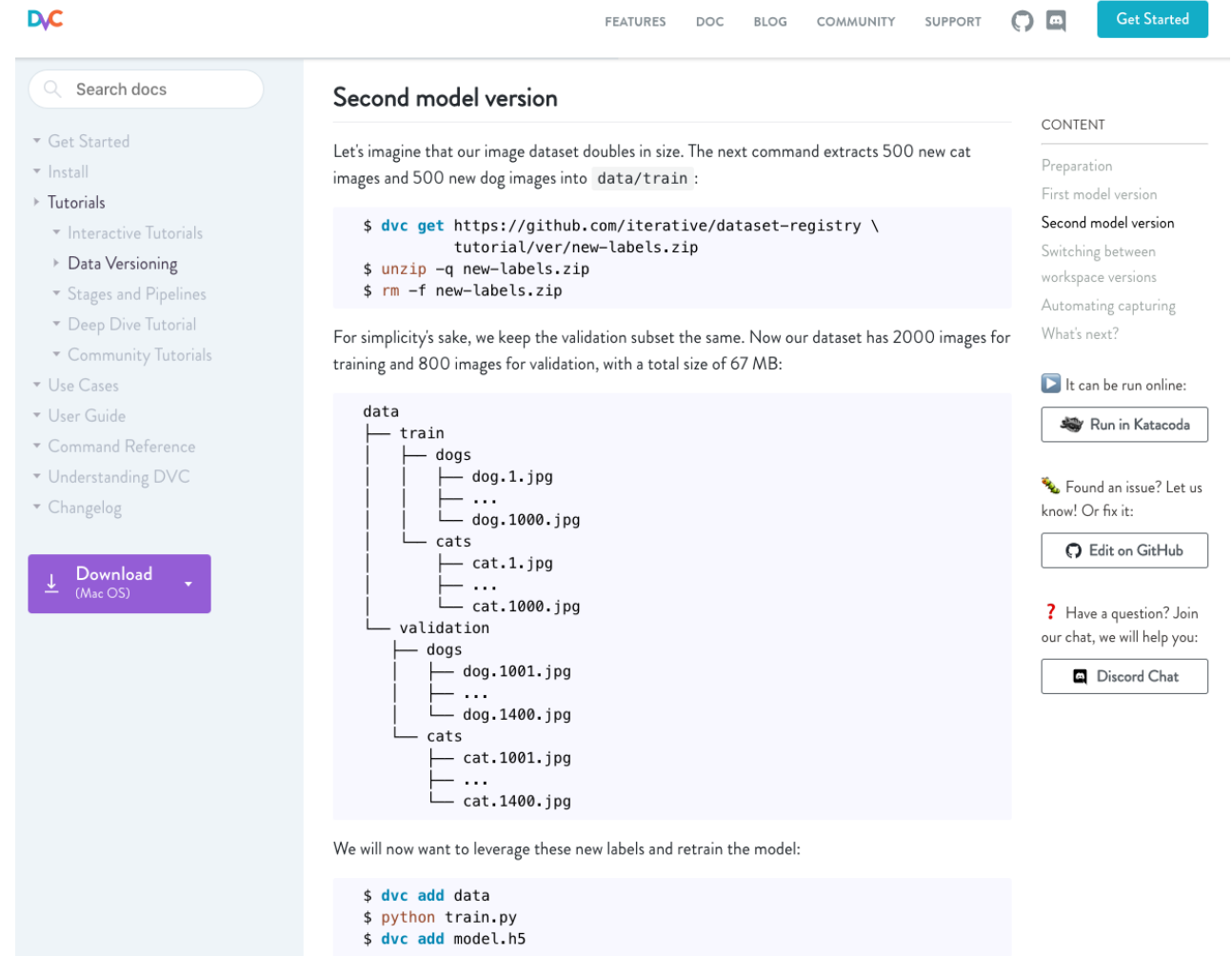
*Note that these query logs contain the data—sensitive!

Data versioning

- Typically, datasets are **too large** for code repositories
- **Example:** I'm training an image classifier using a modified version of ImageNet. I alter some images, add some new ones, etc.
 - I want to keep track of these modifications but can't store it in Git.
 - I never want to re-download the whole dataset

Example: Data Version Control (DVC)

- Connects to remote data storage servers
- Support for adding and removing data files with versioning
- E.g., add/remove an image



The screenshot shows the DVC documentation website. The left sidebar contains a navigation menu with links to 'Get Started', 'Install', 'Tutorials' (including 'Interactive Tutorials', 'Data Versioning', 'Stages and Pipelines', 'Deep Dive Tutorial', and 'Community Tutorials'), 'Use Cases', 'User Guide', 'Command Reference', 'Understanding DVC', and 'Changelog'. A 'Download (Mac OS)' button is also present. The main content area is titled 'Second model version' and includes a paragraph explaining the scenario: 'Let's imagine that our image dataset doubles in size. The next command extracts 500 new cat images and 500 new dog images into data/train :'. Below this is a code block with the following commands:

```
$ dvc get https://github.com/iterative/dataset-registry \
  tutorial/ver/new-labels.zip
$ unzip -q new-labels.zip
$ rm -f new-labels.zip
```

Following the code is another paragraph: 'For simplicity's sake, we keep the validation subset the same. Now our dataset has 2000 images for training and 800 images for validation, with a total size of 67 MB:'. Below this is a directory tree diagram showing the structure of the 'data' directory:

```
data
├── train
│   ├── dogs
│   │   ├── dog.1.jpg
│   │   ├── ...
│   │   └── dog.1000.jpg
│   └── cats
│       ├── cat.1.jpg
│       ├── ...
│       └── cat.1000.jpg
└── validation
    ├── dogs
    │   ├── dog.1001.jpg
    │   ├── ...
    │   └── dog.1400.jpg
    └── cats
        ├── cat.1001.jpg
        ├── ...
        └── cat.1400.jpg
```

Below the directory tree, a paragraph states: 'We will now want to leverage these new labels and retrain the model:'. This is followed by a code block with the following commands:

```
$ dvc add data
$ python train.py
$ dvc add model.h5
```

The right sidebar of the documentation page contains a 'CONTENT' section with links to 'Preparation', 'First model version', 'Second model version' (the current page), 'Switching between workspace versions', 'Automating capturing', and 'What's next?'. At the bottom of the sidebar, there are three buttons: 'Run in Katacoda', 'Found an issue? Let us know! Or fix it: Edit on GitHub', and 'Have a question? Join our chat, we will help you: Discord Chat'.

Data independence

- We often want our systems to support **data independence**
- Changing a training algorithm **shouldn't break** our ability to access and use data, or vice versa
- This is a common problem in “research code” which is written for one specific dataset and one specific representation

L3. Backwards compatibility

- A **backwards-compatible** AI system should be able to reproduce experiments designed for prior versions
- This is necessary for **disentangling** model and data changes
- Modularity and abstraction is critical here
 - **Example:** I design an image classifier for fixed-size images of 640x480 and change it later to 1024x768. Does it still **run** for the former? Does it still produce the **same result**?

L4. Unit/module tests

- An oft-forgotten part of software engineering in ML is **unit testing**
- **Example:** A convolutional neural network in TensorFlow

```
1 def make_convnet(input_image):
2     net = slim.conv2d(input_image, 32, [11, 11], scope="conv1_11x11")
3     net = slim.conv2d(input_image, 64, [5, 5], scope="conv2_5x5")
4     net = slim.max_pool2d(net, [4, 4], stride=4, scope='pool1')
5     net = slim.conv2d(input_image, 64, [5, 5], scope="conv3_5x5")
6     net = slim.conv2d(input_image, 128, [3, 3], scope="conv4_3x3")
7     net = slim.max_pool2d(net, [2, 2], scope='pool2')
8     net = slim.conv2d(input_image, 128, [3, 3], scope="conv5_3x3")
9     net = slim.max_pool2d(net, [2, 2], scope='pool3')
10    net = slim.conv2d(input_image, 32, [1, 1], scope="conv6_1x1")
11    return net
```

- Does it do what it's supposed to?

L4. Unit tests

- It turns out that...

1. This code never crashes, raises an error, or even slows down.
2. This network still trains and the loss will still go down.
3. The values converge after a few hours, but to really poor results, leaving you scratching your head as to what you need to fix.

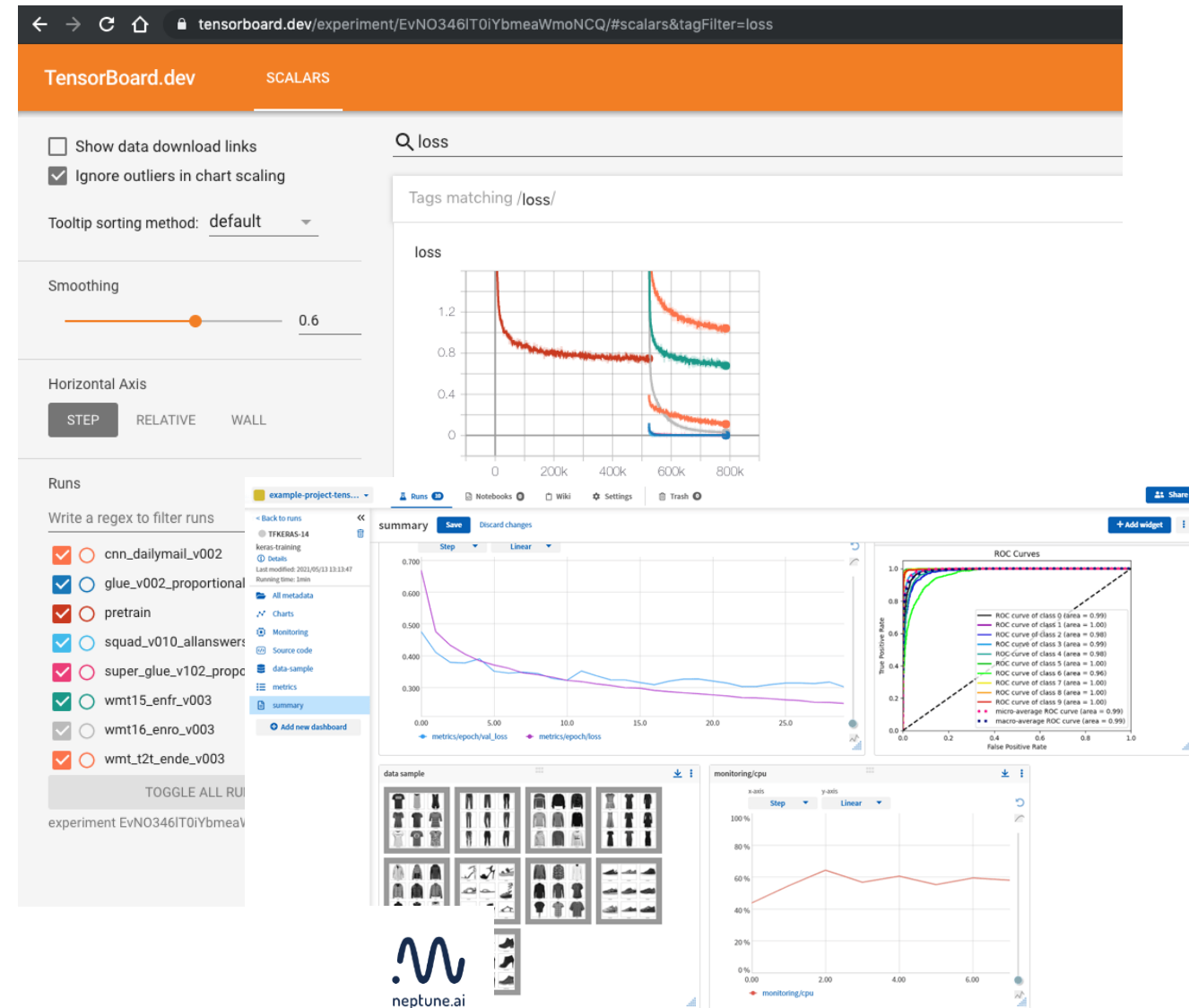
- The network doesn't stack!

```
net = slim.conv2d(input_image, 32, [11, 11], scope="conv1_11x11")  
net = slim.conv2d(input_image, 64, [5, 5], scope="conv2_5x5")  
net = slim.max_pool2d(net, [4, 4], stride=4, scope='pool1')
```

- We see this immediately if **testing gradient values!**

Experiment tracking

- TensorFlow comes with **TensorBoard**, a visualization tool for experiment tracking
- Loads **metrics** stored during training, e.g., losses, embeddings, etc
- These days, many alternatives out there, e.g., Neptune, GuildAI, Tableau, Weights & Biases (wandb)



A short note on hardware

Hardware in AI development

- AI problems are incredibly diverse!
 - Image classification requires huge models and datasets, >1M samples
 - Some clinical datasets have ~1000 samples
- The hardware we need varies as well
 1. Simple prototyping on a laptop
 2. Hyperparameter tuning for non-deep models on CPU clusters
 3. Large-scale reinforcement learning or deep learning on GPU clusters

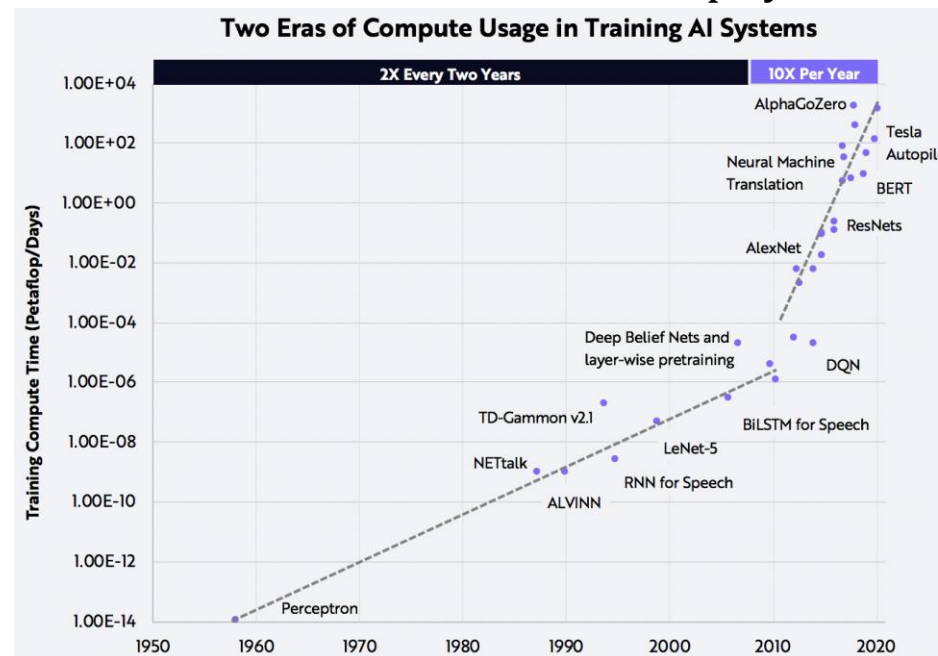
Investments in AI hardware in Sweden

- Chalmers has built a cluster for AI research, Alvis
 - Part of SNIC—Swedish National Infrastructure for Computing
- The Wallenberg program WASP built a NVIDIA SuperPOD called Berzelius in Linköping (also accessible through SNIC)
- Both are very heavily focused on GPU computation
 - Deep learning is the reason!

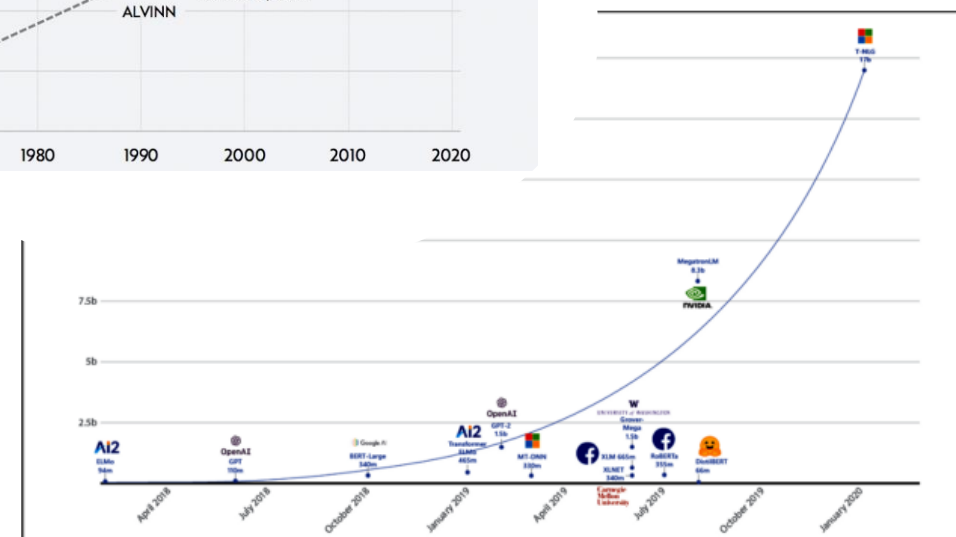
The impact of deep learning

- Deep learning has drastically changed how we use computational resources

Various deep systems



Parameters used in language models

¹OpenAI, ²Soheil Tehranipour

Two main reasons

1. Deep learning is heavily **parallelizable**
 - Main effort is matrix multiplication—well handled by GPUs
 - Training samples may be processed in parallel
 - $\Rightarrow$ We can use more power in shorter wall time

2. Deep learning models can **scale arbitrarily**
 - Almost trivial to make a larger model
 - $\Rightarrow$ We can make use of more data (and more compute)

Thank you!